

UNIT 6: Individual Coding Exercise

1. Introduction

This exercise covers a thread safe banking system written in Python. Several users can deposit, withdraw, check their balance and transfer money on shared accounts at the same time. Based on the exercise statement, “The system must ensure that all operations are thread-safe and avoid common concurrency issues such as race conditions and deadlocks”.

1.1. Application Structure

Class	Responsibility	Method	Ref
BankAccount	Holds account state (owner, number, balance) and performs all thread safe money operation behind a per account lock	Deposit, withdraw, <i>get_balance</i> , <i>transfer</i> , <i>apply_penalty</i> , <i>__str__</i>	2.1
Deposit / Withdraw	Concrete command classes, each wraps one operation so users can queue a list of actions	Execute, Label	2.2
Transaction (ABC)	Abstract base class defining the command interface for an operation on an account	Execute(abstract), Label (abstract)	2.5
TransactionSimulator	Drives the Simulation assigns each user a list of transactions and runs them on one thread per user	User_transaction, run	2.6
TestbankAccount	Unit and integration test suite validating correctness, thread-safety and deadlock freedom.	8 test methods	3.9

2. Implementation

2.1. Class BankAccount

BankAccount holds the owner, a unique account number, the balance, and running totals for deposits, withdrawals and penalties. The constructor also creates one lock per account.

```

class BankAccount:
    def __init__(self, owner, account_number, balance):
        self._owner = owner
        self._account_number = account_number
        self._balance = balance
        self._deposit = 0
        self._withdraw = 0
        self._penalty = 0
        self._lock = threading.Lock() # One lock per account

```

Snippet 2.1 Constructor: one Lock is created per account instance

```

Owner: Jess, Account: 1234, Balance: 570 euro, Deposit: 30, Withdraw: 60, Penalty: 0
-
Owner: James, Account: 7891, Balance: 400 euro, Deposit: 100, Withdraw: 0, Penalty: 0
-
Owner: Lily, Account: 2222, Balance: 45 euro, Deposit: 0, Withdraw: 0, Penalty: 5
-

```

Comment: each account is given its own lock instead of one global lock for the bank. This allows operations on different accounts to run in parallel, while operations on the same account run one at a time. The penalty counter goes up whenever an invalid operation is attempted. See Lily's output, where a refused withdrawal applied a 5 euro penalty.

2.2. Deposit, Withdraw and Balance

Every method that touches the balance runs inside with `self._lock`. The lock is taken on entry and released automatically on exit, so each operation completes in full before another thread can touch the balance.

```

def deposit(self, amount):
    with self._lock:                # only one thread enters at a time
        if amount < 0:
            self.apply_penalty()
            return False
        self._balance += amount
        self._deposit += amount
        return True

def withdraw(self, amount):
    with self._lock:                # locked while withdrawing

        if amount < 0:
            self.apply_penalty()
            return False

```

```

        if self._balance >= amount:
            self._balance -= amount
            self._withdraw += amount
            return True
        self.apply_penalty()    # insufficient funds =penalty
        return False

def get_balance(self):
    with self._lock:          # even reads need locking
        return self._balance

```

Snippet 2.2 All balance touching methods run inside the lock

```

Stephanie, deposit, 200 euros, balance: 1099euro
Stephanie, withdraw, 100 euros, balance: 999euro

Eric, deposit, 60 euros, balance: 1059euro
Eric, withdraw, 20 euros, balance: 1039euro

Melanie, withdraw, 150 euros, balance: 889euro
Melanie, deposit, 250 euros, balance: 1139euro

Final balance (shared): 1139 euro

```

Comment: withdraw only goes through if there is enough money, otherwise it is refused and a penalty is applied, keeping the balance from going negative. Reads are locked too, so `get_balance` cannot return a value mid-update.

2.3. Thread Safety: Preventing Race Conditions

A race condition happens because `self._balance += amount` is really three steps: read, add, write. If two threads interleave those steps, one update is lost. The per account lock stops this by letting only one thread into the critical section at a time.

The scenario 3.5 proves it directly: 100 threads each deposit 10 onto a starting balance of 0. Without the lock the result is unpredictable. With the lock it is exactly 1000.

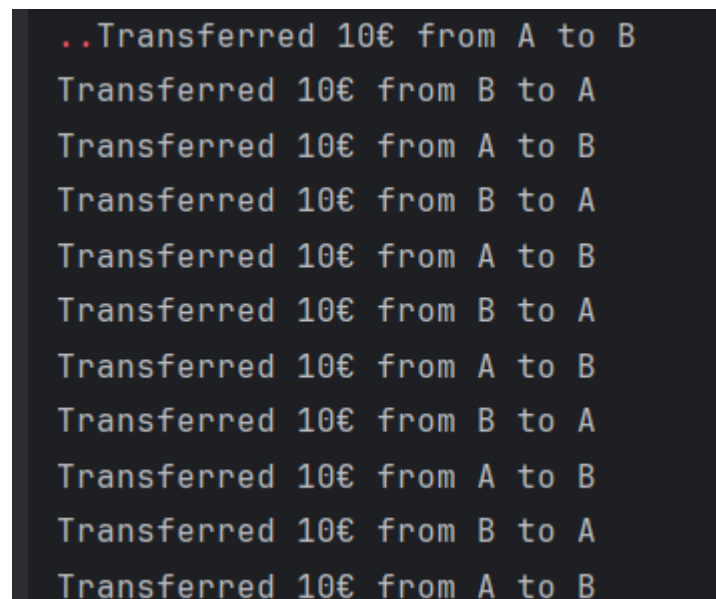
2.4. Transfer and Deadlock Prevention

A transfer needs two locks at once, which is the classic deadlock trap. If thread 1 transfers A to B while thread 2 transfers B to A, each holds one lock and waits for the other forever. My fix is to always lock accounts in order of account number, so every thread takes the locks in the same order and no deadlock can form.

```
def transfer(self, target_account, amount):
    # always lock the account in the same order, deadlock prevention
    if self._account_number < target_account._account_number:
        first, second = self, target_account
    else:
        first, second = target_account, self

    with first._lock:
        with second._lock:
            if self._balance >= amount:
                self._balance -= amount
                target_account._balance += amount
                print(f"Transferred {amount} euro from {self._owner} to
{target_account._owner}")
            return True
```

Snippet 2.4 Lock ordering by account number



```
..Transferred 10€ from A to B
Transferred 10€ from B to A
Transferred 10€ from A to B
Transferred 10€ from B to A
Transferred 10€ from A to B
Transferred 10€ from B to A
Transferred 10€ from A to B
Transferred 10€ from B to A
Transferred 10€ from A to B
Transferred 10€ from B to A
Transferred 10€ from A to B
```

The screenshot shows 100 opposing transfers all finishing. The program never hangs which is the practical proof the rule works.

2.5. The Command Pattern (Transaction Classes)

Each operation is modelled as a Command object so that a user can run a sequence of actions. Transaction is the abstract base class, and Deposit and Withdraw are the concrete

commands. The simulator is given a list of these objects and calls execute on each, without needing to know its concrete type (polymorphism).

```
class Transaction(ABC):
    def __init__(self, amount):
        self.amount = amount

    @abstractmethod
    def execute(self, account):
        pass

    @abstractmethod
    def label(self):
        pass

class Deposit(Transaction):
    def execute(self, account):
        account.deposit(self.amount)

    def label(self):
        return "deposit"

class Withdraw(Transaction):
    def execute(self, account):
        account.withdraw(self.amount)

    def label(self):
        return "withdraw"
```

Snippet 2.5 Command pattern, abstract transaction with concrete deposit/withdraw

2.6. Class TransactionSimulator

The simulator holds one shared account and gives each user a list of Command objects. Then, it creates one thread per user, so they all hit the shared account at the same time and joins every thread before printing the final balance.

```
def run(self):
    # define what each user will do
    users = {
        "Stephanie": [Deposit(200), Withdraw(100)],
        "Eric": [Deposit(60), Withdraw(20)],
        "Melanie": [Withdraw(150), Deposit(250)],
    }

    threads = []
    # create one thread per user
```

```

for user_name , transactions in users.items():
    t = threading.Thread(
        target=self.user_transactions,
        args=(user_name , transactions)
    )
    threads.append(t)

# start all threads (users act at the same time)
for t in threads:
    t.start()

# wait for everyone to finish
for t in threads:
    t.join()

print(f"\nFinal balance (shared): {self._account.get_balance()} euro")

```

Snippet 2.6 One thread per user

```

Stephanie, deposit, 200 euros, balance: 1099euro
Stephanie, withdraw, 100 euros, balance: 999euro

Eric, deposit, 60 euros, balance: 1059euro
Eric, withdraw, 20 euros, balance: 1039euro

Melanie, withdraw, 150 euros, balance: 889euro
Melanie, deposit, 250 euros, balance: 1139euro

Final balance (shared): 1139 euro

```

Comment: with all three users acting on the same account at once, this acts as a concurrency test. The consistent final balance across runs confirms the locking, Command and threading components integrate correctly.

3. Testing and Validation

Two kinds of test were used: unit tests (a single component in isolation) and integration/concurrency tests (several components and many threads at once).

3.1. Scenario Single deposit(unit)

The test confirms a deposit increases the balance by exactly the deposited amount.

```
def test_deposit(self):
    account = BankAccount("Jess", 1234, 100)
    account.deposit(50)
    self.assertEqual(account.get_balance(), 150)
```

Snippet 3.1 test code

✓ test_deposit

0 ms

Comment: Baseline check that deposit adds the right amount, no concurrency involved.

3.2. Scenario Single withdrawal (unit)

The test confirms a valid withdrawal reduces the balance correctly

```
def test_withdraw(self):
    account = BankAccount("Jess", 1234, 100)
    account.withdraw(30)
    self.assertEqual(account.get_balance(), 70)
```

Snippet 3.2 test code

✓ test_withdraw

0 ms

Comment: Checks the deduction path of withdraw when there is enough money

3.3. Scenario Insufficient funds and penalty (unit)

The test confirms an over withdrawal is refused, and a 5 euro penalty is applied.

```
def test_penalty(self):
    account = BankAccount("Lily", 7777, 50)
    account.withdraw(60) # refused, penalty applied
    self.assertEqual(account.get_balance(), 45) #50-5
```

Snippet 3.3 test code

✓ test_penalty

0 ms

Comment: Checks the rule and the penalty branch of withdraw

3.4. Scenario Single transfer (unit)

The test confirms the money has been transferred correctly from one account to another.

```
def test_transfer(self):
    account1 = BankAccount("Jess", 1234, 600)
    account2 = BankAccount("James", 7891, 300)
    account1.transfer(account2, 100)
    self.assertEqual(account1.get_balance(), 500)
    self.assertEqual(account2.get_balance(), 400)
```

Snippet 3.4 test code

✓ test_transfer 0 ms

Comment: Confirms transfer works correctly in the standard case.

3.5. Concurrent deposits, race condition (integration)

The test confirms the per account lock prevents lost updates under 100 simultaneous deposits.

```
def test_thread_safety(self):
    account = BankAccount("Shared", 9999, 0)
    threads = []
    for _ in range(100):
        t = threading.Thread(target=account.deposit, args=(10,))
        threads.append(t)
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    self.assertEqual(account.get_balance(), 1000)
```

Snippet 3.5 test code

✓ test_thread_safety 34 ms

Comment: The result of exactly 1000 on every run is direct evidence that no race condition occurs.

3.6. Scenario Cross transfer deadlock (integration)

The test confirms the lock-ordering rule stops deadlock when two accounts transfer to each other at once.

```
def test_no_deadlock_cross_transfer(self): #two accounts transfer to each other
    concurrently, no deadlock
    a = BankAccount("A", 1, 1000)
    b = BankAccount("B", 2, 1000)
    threads = []
```



```

for _ in range(50):
    threads.append(threading.Thread(target=a.transfer, args=(b, 10)))
    threads.append(threading.Thread(target=b.transfer, args=(a, 10)))
for t in threads:
    t.start()
for t in threads:
    t.join()
self.assertEqual(a.get_balance() + b.get_balance(), 2000) # total unchanged

```

Snippet 3.6 test code

✓ test_no_deadlock_cross_transfer 48 ms

Comment: The test finishes and the total money stay at 2000.

3.7. Mixed concurrent deposit plus withdraw(integration)

The test confirms the balance stays correct when deposits and withdrawals run concurrently under load.

```

def test_concurrent_deposit_withdraw(self): #mixed concurrent deposits + withdrawals.
    final balance must be exact
    account = BankAccount("Shared", 1000, 0)
    threads = []
    for _ in range(100):
        threads.append(threading.Thread(target=account.deposit, args=(10,)))
        threads.append(threading.Thread(target=account.withdraw, args=(5,)))
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    self.assertEqual(account.get_balance(), 500) # 1000 deposited, 500 withdrawn

```

Snippet 3.7 test code

✓ test_concurrent_deposit_withdraw 80 ms

Comment: 1000 deposited minus 500 withdrawn gives 500 on every run, confirming both operations remain correct when run concurrently.

3.8. Scenario Insufficient funds under direct call (unit)

The test confirms withdraw returns False and applies the penalty when funds are too low.

```

def test_withdraw_insufficient_funds(self): #withdrawal refused on insufficient funds
+ penalty applied
    account = BankAccount("Low", 4321, 30)
    self.assertFalse(account.withdraw(100))
    self.assertEqual(account.get_balance(), 25) # 30 - 5 penalty

```

Snippet 3.8 test code

```
✓ test_withdraw_insufficient_funds 0 ms
```

Comment: checks both the return value and the resulting balance for the refusal path

3.9. Overall test run

All eight scenarios run together through `unittest.main()` at the bottom of `main.py`. The combined run reports 8 tests passing, which validates both the individual units and the way they integrate.

```
✓ Test Results 162 ms
  ✓ main 162 ms
    ✓ TestbankAccount 162 ms
      ✓ test_concurrent_deposit_withdraw 80 ms
      ✓ test_deposit 0 ms
      ✓ test_no_deadlock_cross_transfer 48 ms
      ✓ test_penalty 0 ms
      ✓ test_thread_safety 34 ms
      ✓ test_transfer 0 ms
      ✓ test_withdraw 0 ms
      ✓ test_withdraw_insufficient_funds 0 ms
  ✓ 8 tests passed 8 tests total, 162 ms
```

3.10. Validation Summary

Table 3.1 lists every scenario, type, purpose and actual result.

Scenario	Type	Purpose	Expected	Result	Validates
3.1	Unit	Single deposit	Balance 150	Pass	2.2
3.2	Unit	Single withdrawal	Balance 70	Pass	2.2
3.3	Unit	Over-withdraw refused + penalty	Balance 45	Pass	2.2
3.4	Unit	Single transfer	500 / 400	Pass	2.4
3.5	Concurrency	100 concurrent deposits, no race	Balance 1000	Pass	2.3
3.6	Deadlock	Cross transfers, no deadlock	Total 2000, no hang	Pass	2.4
3.7	Concurrency	Mixed deposit/withdraw under load	Balance 500	Pass	2.2/2.3
3.8	Unit	Insufficient funds refusal	False, balance 25	Pass	2.2

4. Conclusion

The system satisfies every requirement: a `BankAccount` class providing `deposit`, `withdraw` and `get_balance`; thread-safety through per-account locks; a `TransactionSimulator` running

concurrent users; deadlock prevention through lock ordering; and a test suite covering both unit and integration testing